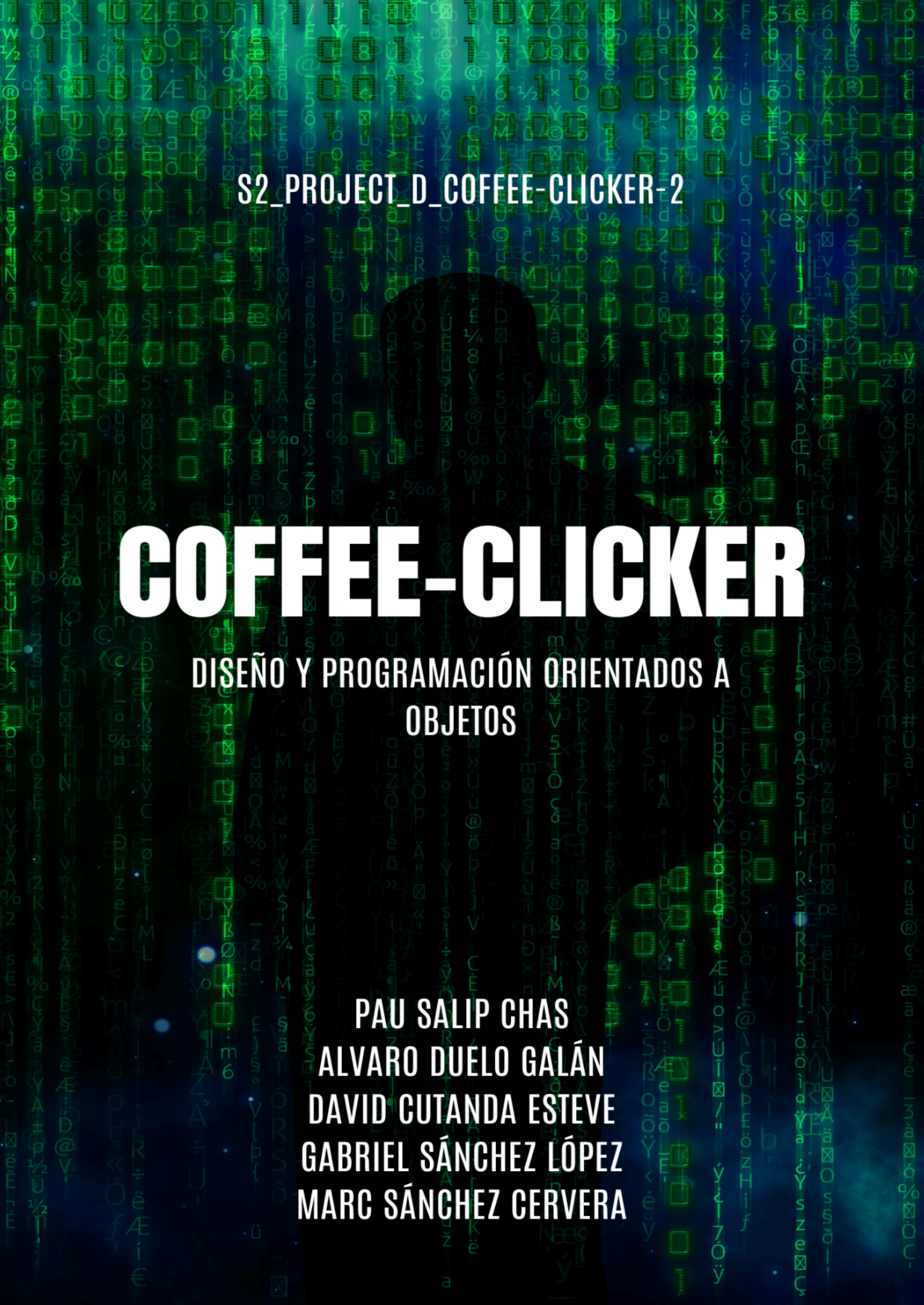
S2_PROJECT_D_COFFEE-CLICKER-2



COFFEE-CLICKER

DISEÑO Y PROGRAMACIÓN ORIENTADOS A OBJETOS

PAU SALIP CHAS
ALVARO DUELO GALÁN
DAVID CUTANDA ESTEVE
GABRIEL SÁNCHEZ LÓPEZ
MARC SÁNCHEZ CERVERA

Índice

1. Resumen de las especificaciones del proyecto 3
2. Diseño de interfaz gráfica de usuario 4
2.1. Mockups (Figma) 4
2.1.1. GUI - Home view 4
2.1.2. GUI - Loginview 5
2.1.3. GUI - Register view 6
2.1.5. GUI - Game view 8
2.1.6. GUI - Statics view 9
2.1.7. GUI - Account view 10
2.1.8. GUI - Errors view 11
2.2. Java (AWT/Swing) 12
2.2.1. Contenedores superiores 12
2.2.2. Contenedores intermedios 14
2.2.3. LayoutManagers 15
2.2.4. Componentes 17
3. Diagrama de clases 20
3.1. Paquete Presentation 32
3.2. Paquete Business 32
3.3. Paquete Persistence 32
4. Metodología de desarrollo 33
5. Costos de tiempo 34
6. Conclusiones 35
7. Bibliografía 36

1. Resumen de las especificaciones del proyecto

Se nos pide crear un juego IDLE similar al famoso cookie clicker pero con cafés en este caso.

Al ejecutar el programa, se leerá un fichero llamado config.json el cual contendrá la información de conexión al puerto de las bases de datos, el nombre de la misma y todos los datos pertinentes para la correcta ejecución del programa.

El programa permite a los usuarios registrarse o iniciar sesión, incluyendo la opción de recuperar la contraseña o iniciar sesión a través de redes sociales. El registro requiere información como nombre de usuario único, correo válido y contraseña que cumpla con ciertos criterios de seguridad. Si el formulario no cumple con los requisitos, se mostrarán mensajes de error. Para iniciar sesión, se necesita el nombre de usuario o correo utilizado en el registro y la contraseña asociada. Las credenciales serán validadas y se permitirá el acceso si son correctas; de lo contrario, se mostrará un mensaje de error genérico.

El programa debe permitir que los usuarios de la aplicación deben tener la opción de cerrar sesión desde cualquier pantalla, lo que los devolverá a la pantalla de registro o inicio de sesión. Además, conforme al RGPD (Art. 17), se debe permitir a los usuarios eliminar su cuenta, lo que implica borrar toda su información del sistema, incluyendo la base de datos y archivos. Estas funciones pueden ser agrupadas en un panel de gestión de cuenta accesible desde cualquier parte del programa.

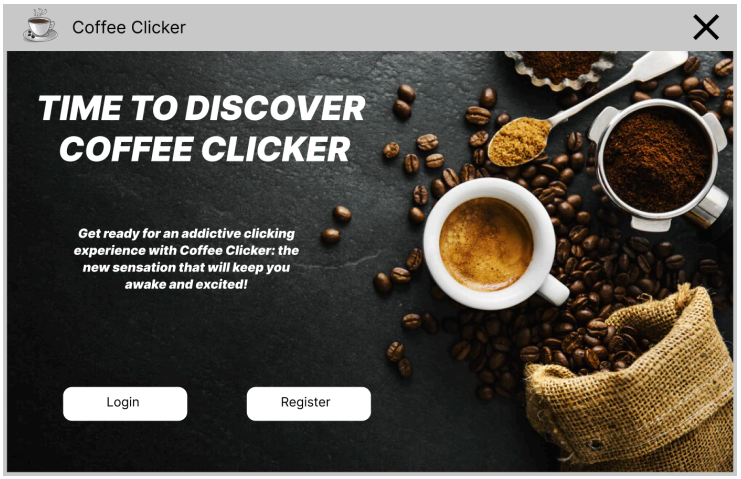
Una vez autenticado, el usuario puede iniciar una nueva partida en el juego, donde se cargarán todos los componentes y dinámicas. Durante la partida, puede decidir finalizarla o salir del juego, almacenando los datos para futuras recuperaciones. Si hay una partida en curso, se mostrará en lugar de la opción de iniciar una nueva. El objetivo del juego, un gestor de recursos, es crear la mayor cantidad de recursos en el menor tiempo posible, sin un final definido, dejando la decisión de dejar de jugar al usuario. Se utiliza el término "cafés" para referirse al recurso, aunque se puede cambiar libremente. Además, se emplean generadores que automáticamente crean recursos a intervalos específicos.

Tras iniciar partida se debe de mostrar la vista del juego dónde sea visible todos los elementos del mismo. Debe de aparecer también un contador de cafés que se incremente cada vez que el jugador le dé al botón manualmente o cuando un generador genere un cafe. Tendrá que haber un botón para compra manual de cafés que incremente la cantidad de cafés en una unidad y luego pueda ser mejorado. Se debe permitir la compra de generadores a cambio de cafés y al comprar se debe de poner de manera automática a generar. Se deberá de mostrar una tabla con el nombre del generador, coste base, producción base e incremento de costes. También deberá de haber una shop de upgrades que permita la adquisición de mejoras al jugador. Por último, habrá una pestaña de estadísticas que consiste en un gráfico de líneas que mostrará la evolución de las partidas finalizadas.

2. Diseño de interfaz gráfica de usuario

2.1. **Mockups (Figma)**: Enlace al figma

2.1.1. GUI - Home view:

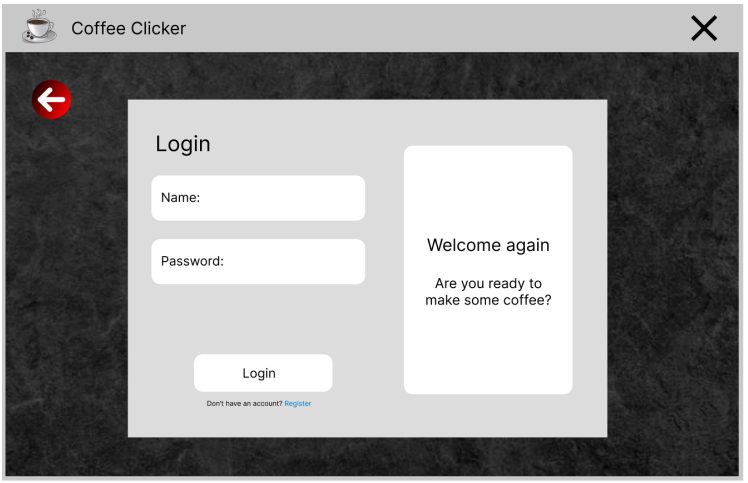


Esta pantalla es la pantalla inicial que aparece al entrar en Coffee Clicker, home view permite al usuario hacer login y registrarse si no tiene una cuenta creada y también mostrará un mensaje de bienvenida.

La Home View está compuesta por dos JButtons siendo estos el botón de Login y el de Register. Adicionalmente, tiene dos JLabels que son el mensaje de “TIME TO DISCOVER COFFEE CLICKER” y la descripción justo por debajo de este. También, esta view tiene un BufferedImage que sería la imagen de fondo con los granos de café.

Esta pestaña se ha hecho lo más minimalista posible para que sea intuitiva, se han añadido los botones abajo y juntos para que sea más fácil de navegar. Justo arriba, los textos de bienvenida que es dónde los ojos van de manera automática y para que la composición quede equilibrada se ha añadido una imagen en el lado vacío, siendo este el derecho.

2.1.2. GUI - Loginview:

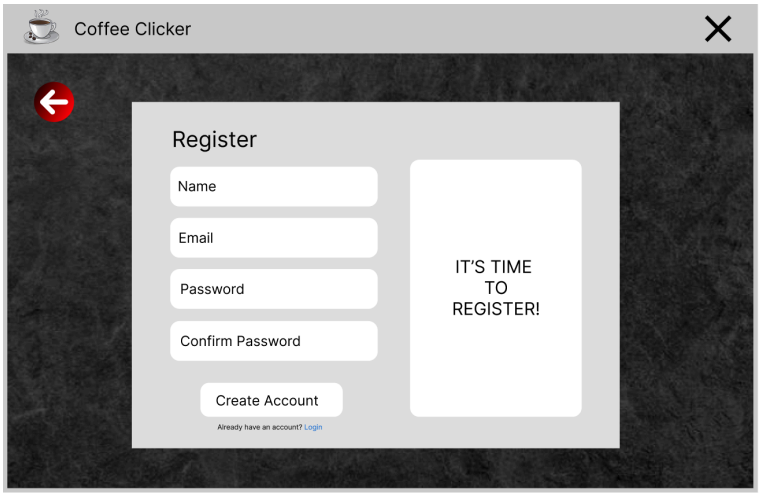


Esta es la pestaña del login la cual permitirá al usuario introducir su nombre de cuenta y contraseña y si existe una cuenta con esos credenciales se entrará en dicha cuenta. También se puede volver a la pantalla anterior y hay una opción por debajo de login que permite registrarse en caso de no tener una cuenta ya creada.

La Loginview está compuesta por tres JButtons, el de la flecha de retroceso que lleva a la pestaña anterior, el de login que se encuentra abajo del todo y el de justo debajo que es el de registro. También, tiene un JTextField para introducir el username y un JPasswordField justo debajo para introducir la contraseña. Tiene una imagen de fondo que es una BufferedImage. El rectángulo blanco ha sido creado mediante un JPanel y el rectángulo del texto de welcome again también es un JPanel. El texto de “Don’t have an account” es un JPanel y el resto de textos que aparecen en pantalla serán JLabels.

Hemos dividido en dos esta pestaña, por un lado los mensajes de bienvenida y por otro el proceso del login. Esto se ha hecho para segmentar y ordenar la pestaña y se ha puesto todas las opciones de login dentro de un recuadro para resaltar la. Por último se ha añadido un fondo de color cafe para resaltar el rectangulo y va con la estetica de la app.

2.1.3. GUI - Register view:



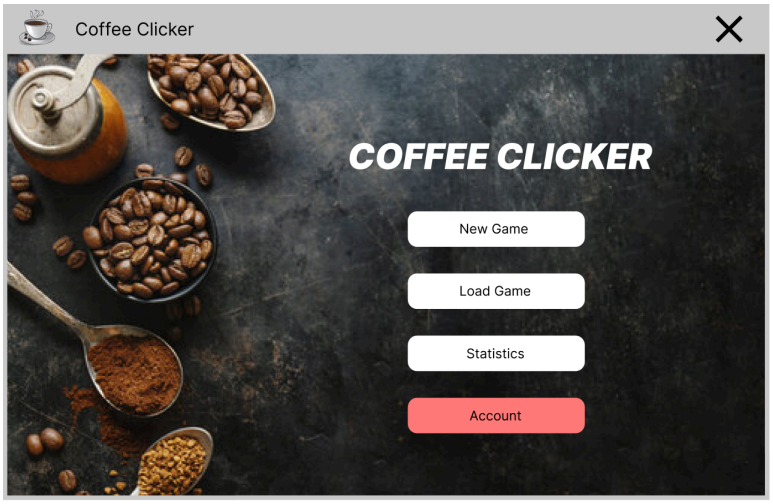
Esta es la pestaña de register que permite al usuario registrarse introduciendo el nombre de usuario que desee, mail, contraseña y deberá confirmar la contraseña. Se muestra un mensaje incentivando al registro y por supuesto, un botón para retroceder. En caso que un usuario ya registrado haya accedido a esta pestaña por error, tendrá la posibilidad de acceder directamente desde este View a la que te permite hacer el Login.

La Registerview está compuesta por tres JButtons, el de la flecha de retroceso que lleva a la pestaña anterior, el de create account que se encuentra abajo del todo y el de justo debajo que es el de login. Usa dos JTextField para introducir tanto el username como email, luego un JPasswordField justo debajo para introducir la contraseña y debajo de este otro JPasswordField para la confirmación.

Tiene una imagen de fondo que es una BufferedImage. El rectángulo blanco ha sido creado mediante un JPanel al igual que el rectángulo del texto “It’s time to register!”. El texto de “Already have an account?” es un JPanel y el resto de textos que aparecen en pantalla serán JLabels.

Hemos dividido en dos esta pestaña, por un lado los mensajes de bienvenida y por otro el proceso de registrarse. Esto se ha hecho para segmentar y ordenar la pestaña y se ha puesto todas las opciones de login dentro de un recuadro para resaltar la. Por último se ha añadido un fondo de color cafe para resaltar el rectangulo y va con la estetica de la app.

2.1.4. GUI - Menu view:

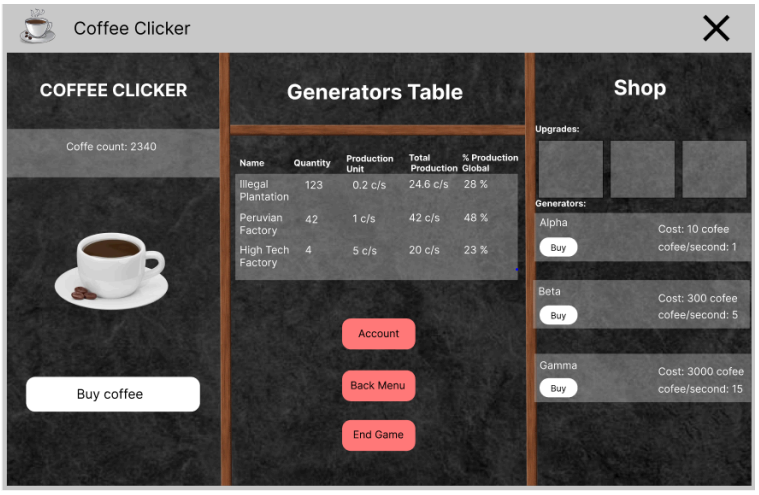


Esta es la pestaña del menú principal. Desde esta podremos acceder a varias funcionalidades como ahora “New Game”, para iniciar una partida en caso que no haya ninguna en curso,“Load Game” para cargar la partida en curso,“Statistics” para ver las estadísticas y por último “Account” para acceder a lla administración de la cuenta del usuario.

El Menú view consiste en un JFrame que tendrá como título “COFFEE CLICKER” y un ImageIcon de fondo. También, tendrá una CardLayout y un JPanel que serán los botones que llevarán a las distintas views de cargar una partida, ver estadísticas o ver cuenta.

Está diseñado y compuesto de tal forma que resalta el contenido más relevante de la imagen de fondo a parte de cumplir con una estética francamente atractiva y elegante. De esta forma, tendremos las opciones ligeramente a la derecha junto el título del videojuego y a la izquierda el segmento de la imagen fondo que consideramos más relevante.

2.1.5. GUI - Game view:

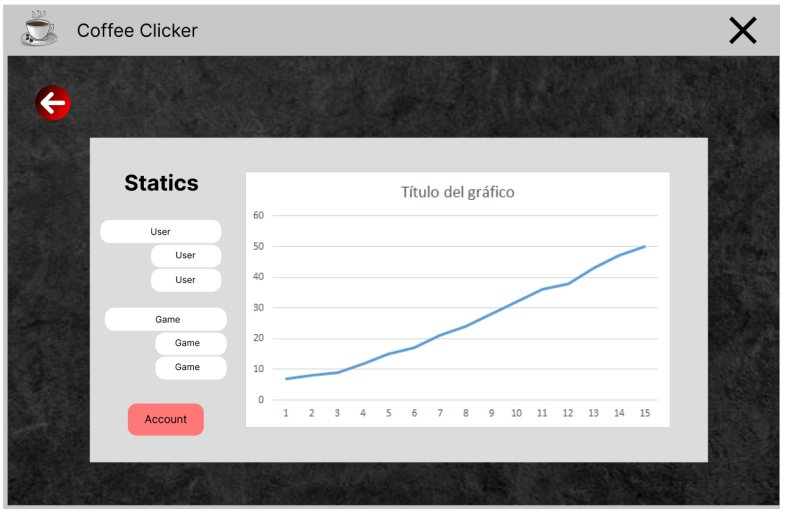


Esta es la pestaña del juego, segmentada por 3 zonas diferenciadas. La primera de todas, a la izquierda tenemos el título del juego “Coffee Clicker” el contador de cafés que hemos comprado, en el cual se encuentra también el botón de comprar cafés “Buy coffee”. El contador irá aumentando manualmente por cada click que hagamos en el botón, en caso que tengamos activa alguna mejora o generador este irá incrementando de forma automática y adicional a los clicks.
Después en la parte central tenemos la tabla de generadores donde por cada uno de los generadores comprados, podremos observar su nombre y la cantidad de generadores comprados de cada uno de ellos, la producción por unidad de cada uno, la producción total de cada uno y el porcentaje de la producción global que tienen todos los generadores respecto al total. Además debajo de la tabla se puede observar tres botones diferentes, el primero para ir a la cuenta del usuario, el segundo botón para ir otra vez al menú del juego, y el tercer botón para finalizar la partida.
Por último, en la parte derecha de la view tenemos la tienda del juego. En éste apartado tendremos los botones con imágenes que, al pasar por encima podremos ver información sobre cada una de las diferentes mejoras. Es decir, el nombre de la mejora, la producción y por último el coste, el cual se actualiza cada vez que la mejora es comprada.
Además de las mejoras en la parte de abajo tenemos los generadores. A cada generador le corresponde una imagen representativa, un nombre, el coste del generador y por último la cantidad de cafés que es capaz de producir por segundo. En total tendremos 3 generadores. Además tenemos la opción de comprar ese generador haciendo click en el botón “Buy”. En caso de no tener suficientes cafés, nos aparecerá un mensaje de error.

El Gameview está compuesto por dos JLabels siendo estos los de la tabla de generadores, el de “Coffee clicker” y el de “Shop”. Tenemos varios JButtons siendo estos los de compra de mejoras, los de comprar café y los de volver a la cuenta, volver al menú y finalizar juego. Tenemos diversos JPanels que dividen las zonas, tenemos ImageIcons para el fondo y JLabel para el número de cafés. Otros ImageIcons serán las imágenes de las upgrades. Para la tabla de información se usa un JPanel y JLabels para introducir los datos.

La game view se ha dividido en tres secciones para que sea más fácil localizar las cosas y se ha empleado lo mínimo posible para que sea más fácil de navegar.

2.1.6. GUI - Statics view:

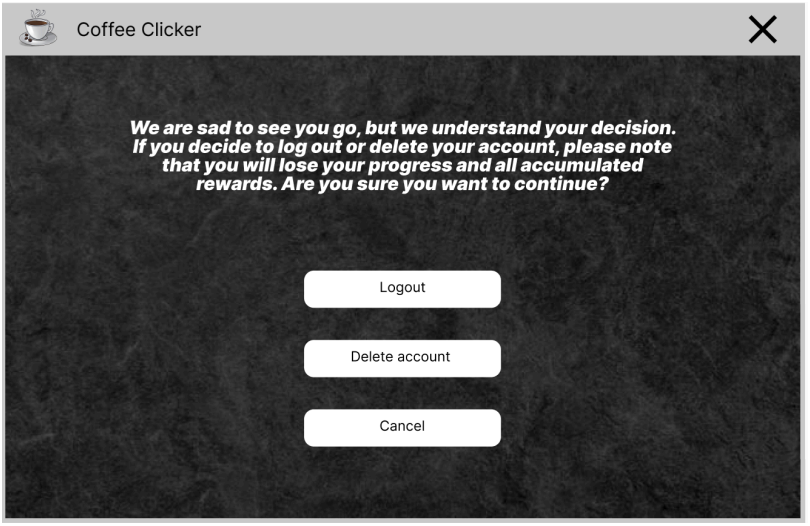


Esta pestaña es la de las estadísticas, aquí se podrá observar mediante una gráfica la evolución de las partidas finalizadas. Además tiene la opción de poder escoger al usuario y la partida que se quiera ver. De cualquiera de las formas, el gráfico siempre mostrará en su eje horizontal el tiempo transcurrido desde el inicio al final de la partida en minutos y en el eje vertical la cantidad de cafés disponibles en aquel instante de tiempo. De esta forma podremos ver la evolución de la partida. Esta información se irá actualizando en tiempo real.

En la Statisticsview hay varios JButtons siendo estos el de retroceso y el de account. Tenemos una BufferedImage que sería la imagen de fondo, un JLabel que es el título “Statics”, un JPanel que es el rectángulo blanco, varias JComboBox para “User” y “Game” y un JPanel para la gráfica.

La pestaña se ha agrupado la información en el centro para que sea más visible y sea lo primero que el usuario se fije.

2.1.7. GUI - Account view:



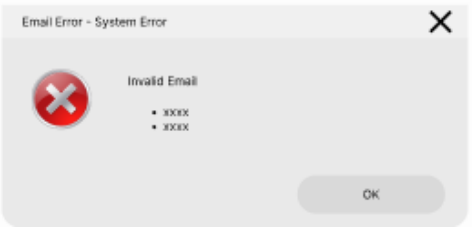
Esta es la pestaña de gestión usuario, donde se muestra un texto informando al usuario de lo que puede pasar si cierra la sesión o elimina la cuenta. Además de este texto, también da tres opciones mediante tres botones, el primero sirve para cerrar sesión, donde al hacer click, nos saldrá una nueva pestaña donde nos preguntará si realmente queremos cerrar la sesión. Si le damos a “si” nos redirigirá al menú principal. El segundo botón es para eliminar la cuenta que tenemos, donde al hacer clic nos saldrá la misma pestaña con la misma pregunta pero ahora sobre si realmente queremos eliminar la cuenta. Al hacer clic en “si” nos redirigirá al menú principal. Y por último tenemos el tercer botón de cancelar, donde al hacer clic en “cancel” nos redirigirá a las distintas views dependiendo de donde estábamos antes de hacer clic en “Account”.

La Accountview está compuesta por una BufferedImage que sería el fondo de la view, varios JButtons que sería los de “Logout”, “Delete account” y “Cancel” y un JLabel que es la descripción que está arriba de los botones.

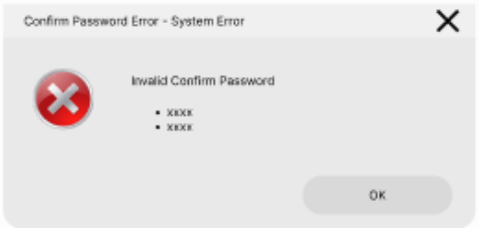
Los botones se han agrupado en medio junto con el texto para que sean visibles y estén ordenados, la imagen de fondo se ha seleccionado para hacer contraste y resaltar los botones y texto.

2.1.8. GUI - Errors view:









En esta parte podemos ver las distintas pestañas con los distintos mensajes de error que nos pueden aparecer.
Tenemos una pestaña para el error del nombre de usuario, el error del correo, el error de la contraseña y por último otra para el error de confirmación de contraseña, en caso que éstas sean incorrectas. En cada uno de ellos nos saldrá el mensaje del error, el error en cuestión y un botón de confirmación “ok” para luego poder arreglar ese error que nos ha salido.

Todos los mensajes de error están compuestos por un JOptionPane los cuales contienen el nombre del error, un icono, texto elaborando el error y dos botones, uno para cerrar el error y otro de ok para aceptar.

Esto ha sido hecho de esta forma ya que es el formato de error pop-up standard y es sencillo averiguar a qué se debe el error con este formato.

2.2. Java (AWT/Swing):

2.2.1. Contenedores superiores

Estos son la base para la creación y la construcción del GUI. En esta categoría se encuentran los JFrame y ciertas operaciones.

JFrame:
El JFrame proporciona métodos y componentes para diseñar y manipular ventanas, como botones, etiquetas, paneles, etc.
Creamos un JFrame que será la ventana que se muestre para cada view en el Mainview. Para hacer esto primero se crea una instancia frame de la clase JFrame.

```
private final JFrame frame;
```

Después, el siguiente código define un constructor para la clase `MainView` en Java, que recibe instancias de varias vistas como parámetros. Dentro del constructor, se crea una instancia de `ImageIcon` con la ruta de una imagen, luego se crea un nuevo `JFrame` con el título "Coffee Clicker" y se establece el ícono de la ventana como la imagen cargada. Posteriormente, se llama al método “createMainPanel()” con las instancias de las vistas recibidas como argumentos para construir y organizar los elementos de la interfaz gráfica dentro de la ventana.

```
1 usage
public MainView(HomeView homeView, LoginView loginView, RegisterView registerView, MenuView menuView,
                AccountView accountView, GameView gameView, StatisticsView staticsView) {
    ImageIcon coffeeIcon = new ImageIcon( filename: "Images/CoffeeClicker_CoffeeCup.jpg");
    this.frame = new JFrame( title: "Coffee Clicker");
    this.frame.setIconImage(coffeeIcon.getImage());

    createMainPanel(homeView, loginView, registerView, menuView, accountView, gameView, staticsView);
}
```

Por último, mediante el siguiente código, creamos una clase que es una vista en Java, que configura y muestra una ventana de JFrame. Primero, agrega la instancia actual de la vista a la ventana, luego establece el tamaño de la ventana como 1000 x 700 píxeles, el título como "Coffee Clicker", y configura la operación de cierre de la ventana para que termine la aplicación cuando se cierra. Luego, centra la ventana en la pantalla y la hace visible para el usuario.

```
    public void initialize() {
        frame.add(this);
        frame.setSize(new Dimension( width: 1000,  height: 700));
        frame.setTitle("Coffee Clicker");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```

Operaciones:

Los métodos de JFrame en Java se utilizan para configurar y manipular ventanas de la GUI. Estos métodos pueden establecer propiedades como tamaño y título de la ventana, gestionar operaciones de cierre y visibilidad, así como también controlar la ubicación de la ventana en la pantalla.
Hemos empleado para la práctica los siguientes:

El método setSize se utiliza para establecer el tamaño de la ventana.

```
frame.setSize(new Dimension( width: 1000,  height: 700));
```

El método setVisible se utiliza para hacer visible o invisible la ventana en la pantalla.

```
frame.setVisible(true);
```

El método setTitle que se utiliza para poner un título en la ventana.

```
frame.setTitle("Coffee Clicker");
```

El método setDefaultCloseOperation establece la acción que se debe realizar cuando el usuario cierra la ventana.

```
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

El método setLocationRelativeTo establece la posición de la ventana en la pantalla.

```
frame.setLocationRelativeTo(null);
```

2.2.2. Contenedores intermedios

Estos nos permiten agrupar los componentes visuales del programa y se encuentran dentro de un componente superior. Aquí, hallaremos a los JPanels y JScrollpanes.

JPanel:
Esta clase se utiliza para crear áreas dentro de una ventana de la GUI donde se pueden colocar otros componentes, como botones, etiquetas y campos de texto.

Declaración de JPanel:

```
JPanel rightPanel = new JPanel(new GridBagLayout());
```

Implementación de JPanel, la función createRightPanel() en Java crea y devuelve un JPanel con un diseño GridBagLayout y un fondo de color gris claro. Dentro de este panel, se agregan dos etiquetas de texto JLabel: una que dice "Welcome back" con un tamaño de fuente grande y centrado, y otra que contiene el mensaje "Are you ready to make some coffee?" con un tamaño de fuente más pequeño y también centrado.

```
private JPanel createRightPanel() {
    JPanel rightPanel = new JPanel(new GridBagLayout());
    rightPanel.setBackground(Color.lightGray);

    // FIRST TEXT
    JLabel text1 = new JLabel( text: "Welcome back");
    text1.setFont(new Font( name: "Arial", Font.BOLD, size: 24));
    text1.setHorizontalAlignment(SwingConstants.CENTER);
    GridBagConstraints gbc1 = new GridBagConstraints();
    gbc1.gridx = 0;
    gbc1.gridy = 0;
    gbc1.insets = new Insets( top: 0, left: 0, bottom: 10, right: 0);
    rightPanel.add(text1, gbc1);

    // SECOND TEXT
    JLabel text2 = new JLabel( text: "<html><center>Are you ready to<br>make some coffee?</center></html>");
    text2.setFont(new Font( name: "Arial", Font.PLAIN, size: 18));
    text2.setHorizontalAlignment(SwingConstants.CENTER);
    GridBagConstraints gbc2 = new GridBagConstraints();
    gbc2.gridx = 0;
    gbc2.gridy = 1;
    gbc2.insets = new Insets( top: 10, left: 0, bottom: 0, right: 0);
    rightPanel.add(text2, gbc2);

    return rightPanel;
}
```

JScrollpane:
Es una clase en Java que se utiliza para proporcionar barras de desplazamiento a componentes que son demasiado grandes para ser completamente visibles dentro de un contenedor, como un JPanel o un JTextArea.

```
JScrollPane scrollPane = new JScrollPane(table);
scrollPane.setPreferredSize(new Dimension( width: 130, height: 245));
```

2.2.3. LayoutManagers

Se utilizan para organizar los componentes de la GUI dentro de los contenedores, como paneles y marcos. Estos gestores de diseño son responsables de determinar la posición y el tamaño de los componentes en función de reglas específicas de diseño. Aquí, hallaremos los BorderLayouts, BoxLayouts, FlowLayouts, GridBagLayout, GridLayout y CardLayout.

BorderLayout:
Es un gestor de diseño en Java que se utiliza para organizar los componentes de una GUI en cinco regiones predefinidas.

```
public AccountView() {
    //System.out.println("\nACCOUNT");

    // DEFINE ATTRIBUTES
    logoutButton = new JButton( text: "Logout");
    deleteAccountButton = new JButton( text: "Delete Account");
    cancelButton = new JButton( text: "Cancel");
    description = new JLabel( text: "<html><center>We are sad to see you go, but we understand your decision.<br>" +
            "If you decide to log out or delete your account, please note<br>that you will lose your progress and al
            "accumulated<br>rewards. Are you sure you want to continue?</center></html>");

    setLayout(new BorderLayout());
    backgroundImage = new ConfigImage(imagePath);
    add(backgroundImage, BorderLayout.CENTER);

    // TEXT
    description.setForeground(Color.WHITE);
    Font descriptionFont = new Font( name: "Arial", Font.BOLD, size: 24);
    description.setFont(descriptionFont);
    description.setHorizontalAlignment(SwingConstants.CENTER);
    JPanel descriptionPanel = new JPanel(new BorderLayout());
    descriptionPanel.setOpaque(false);
    descriptionPanel.setBorder(BorderFactory.createEmptyBorder( top: 100, left: 50, bottom: 50, right: 50));
    descriptionPanel.add(description, BorderLayout.NORTH);
    backgroundImage.add(descriptionPanel, BorderLayout.CENTER);
```

BoxLayout:
Es un gestor de diseño en Java que se utiliza para organizar los componentes de una GUI en una sola fila o columna.

```
private static JPanel createLeftPanel() {
    JPanel leftPanel = new JPanel();
    leftPanel.setBackground(Color.LIGHT_GRAY);
    leftPanel.setLayout(new GridLayout( rows: 8, cols: 1));

    // TITLE "STATISTICS"
    title.setFont(new Font( name: "Arial", Font.BOLD, size: 16));
    title.setForeground(Color.BLACK);
    title.setBorder(BorderFactory.createEmptyBorder( top: 60, left: 10, bottom: 10, right: 0));
    JPanel titlePanel = new JPanel();
    titlePanel.setBackground(Color.LIGHT_GRAY);
    titlePanel.setLayout(new BoxLayout(titlePanel, BoxLayout.X_AXIS));
    titlePanel.add(Box.createHorizontalGlue());
    titlePanel.add(title);
    titlePanel.add(Box.createHorizontalGlue());
    leftPanel.add(titlePanel);

    // BLANK SPACE
    JPanel blankPanel1 = new JPanel();
    blankPanel1.setBackground(Color.LIGHT_GRAY);
    leftPanel.add(blankPanel1);
```

FlowLayout:
Es un gestor de diseño en Java que se utiliza para organizar los componentes de la GUI en una única fila o columna, donde los componentes se colocan secuencialmente uno al lado del otro en el orden en que fueron agregados. Es el administrador de diseño por defecto del JPanel.

```
gbc.gridx = 0;
gbc.gridy = 4;
JPanel row5Panel = new JPanel(new FlowLayout(FlowLayout.LEFT));
```

GridBagLayout:
Es un gestor de diseño en Java que se utiliza para organizar los componentes de una GUI en una cuadrícula de filas y columnas.

```
private void setupLeftPanel() {  1 usage
    JPanel leftPanel = new JPanel();
    Color customColor = new Color( r: 210,  g: 210,  b: 210);
    leftPanel.setBackground(customColor);
    leftPanel.setLayout(new GridBagLayout());
```

GridLayout:
Es un gestor de diseño en Java que se utiliza para organizar los componentes de una GUI en una cuadrícula regular de filas y columnas.

```
    JPanel buttonPanel = new JPanel(new GridLayout( rows: 3,  cols: 1,  hgap: 0,  vgap: 40));
    buttonPanel.setOpaque(false);
    buttonPanel.setBorder(BorderFactory.createEmptyBorder( top: 0,  left: 200,  bottom: 100,  right: 200));
    Dimension buttonSize = new Dimension( width: 250,  height: 50);
    logoutButton.setPreferredSize(buttonSize);
    deleteAccountButton.setPreferredSize(buttonSize);
    cancelButton.setPreferredSize(buttonSize);
```

CardLayout:
Es un gestor de diseño en Java que se utiliza para organizar los componentes de una GUI de manera que solo se muestre un componente a la vez, es decir, permite que se puedan ver varios componentes en espacio tiempos distintos.

Declaración:

```
public class MainView extends JPanel {  4 usages
    private CardLayout cardLayout;  9 usages
    private JPanel cardPanel;  16 usages
    private final JFrame frame;  8 usages
```

Uso de CardLayout:

```
public void showLogin() {  1 usage
    cardLayout.show(cardPanel, LOGIN);
}
```

2.2.4. Componentes

Los componentes son los elementos visuales de la GUI. Entre ellos están los JTextFields, JLabels, JOptionPanes, JButtons, JComboBoxs, JScrollPanes y BufferedImage.

JTextField:
Es un componente en Java Swing que se utiliza para permitir que los usuarios ingresen y editen texto de una sola línea en la GUI.

```
public LoginView() {  1 usage
    // DEFINE ATTRIBUTES
    registerButton = new JButton( text: "REGISTER");
    loginButton = new JButton( text: "LOGIN");
    usernameField= new JTextField( columns: 15);
    passwordField = new JPasswordField( columns: 15);
```

JLabel:
Es un componente en Java Swing que se utiliza para mostrar texto o imágenes en la GUI sin permitir la interacción del usuario con el contenido mostrado.

```
// ROW 2: DISPLAY COFFEE COUNTER
gbc.gridy++;
coffeeCounterLabel.setText("Coffee Count: " + coffeesAvailable);
coffeeCounterLabel.setHorizontalAlignment(SwingConstants.CENTER);
leftPanel.add(coffeeCounterLabel, gbc);
```

JOptionPane:
Es una clase en Java que proporciona una manera simple de crear y mostrar cuadros de diálogo predefinidos en una aplicación Swing.

```
public boolean showLogoutAccount() {  1 usage
    String windowTitle = "Logout Account";
    String errorMessage = "Are you sure you want to log out?";
    int option = JOptionPane.showConfirmDialog( parentComponent: null, errorMessage, windowTitle, JOptionPane.YES_NO_OPTION);
    return option == JOptionPane.YES_OPTION;
}
```

JButton:
Es un componente en Java Swing que se utiliza para representar un botón en una GUI.

```
public AccountView() {  1 usage
    // DEFINE ATTRIBUTES
    logoutButton = new JButton( text: "Logout");
    deleteAccountButton = new JButton( text: "Delete Account");
    cancelButton = new JButton( text: "Cancel");
```

JComboBox:
Es un componente en Java Swing que se utiliza para crear un menú desplegable.

```
// DROPDOWN USER
String[] user = {"User 1", "User 2", "User 3"};
JComboBox<String> userDropdown = new JComboBox<>(user);
userDropdown.setFont(new Font( name: "Arial", Font.PLAIN,  size: 12));
userDropdown.setOpaque(false);
userDropdown.setBorder(BorderFactory.createEmptyBorder( top: 10,  left: 10,  bottom: 10,  right: 10));
leftPanel.add(userDropdown);
```

JScrollPane:
Es un contenedor especial en Java Swing que se utiliza para agregar barras de desplazamiento a otros componentes, como paneles, tablas y áreas de texto.

```
// Create table
JTable table = new JTable(transposedData, rowNames);
table.setRowHeight(55);
JScrollPane scrollPane = new JScrollPane(table);
scrollPane.setPreferredSize(new Dimension( width: 130,  height: 245));
GridBagConstraints tableConstraints = new GridBagConstraints();
tableConstraints.gridx = 0;
tableConstraints.gridy = 1;
tableConstraints.gridwidth = 1;
tableConstraints.fill = GridBagConstraints.BOTH;
tableConstraints.insets = new Insets( top: 10,  left: 10,  bottom: 10,  right: 10);
centerPanel.add(scrollPane, tableConstraints);
```

BufferedImage:
Se utiliza para representar y manipular imágenes en formato de mapa de bits.

```java
public class ConfigImage extends JPanel {

    6 usages
    private BufferedImage backgroundImage;

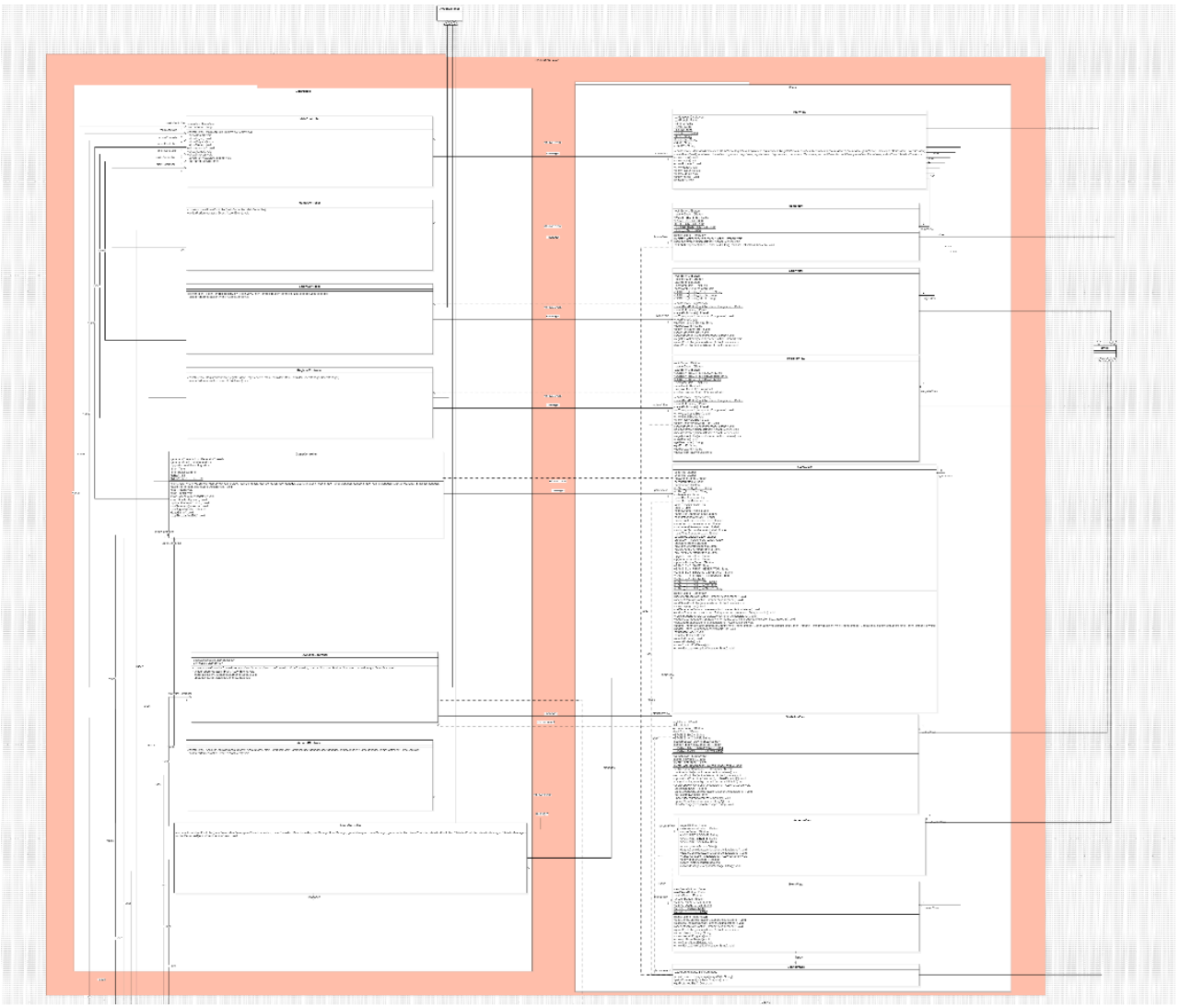
    /**
     * Constructor that loads the image from a file.
     * @param imagePath The path of the image file.
     */
    6 usages
    public ConfigImage(String imagePath) {
        try {
            backgroundImage = ImageIO.read(new File(imagePath));
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    /**
     * Method to draw the image on the component.
     * @param graphics The Graphics context in which to paint.
     */
    @Override
    protected void paintComponent(Graphics graphics) {
        super.paintComponent(graphics);
        if (backgroundImage != null) {
            Image scaledImage = backgroundImage.getScaledInstance(getWidth(), getHeight(), Image.SCALE_SMOOTH);
            graphics.drawImage(scaledImage,  x: 0,  y: 0,  observer: this);
        }
    }
    /**
     * Method to obtain the preferred dimension of the component.
     * @return The preferred size of the component.
     */
    @Override
    public Dimension getPreferredSize() {
        if (backgroundImage != null) {
            return new Dimension(backgroundImage.getWidth(), backgroundImage.getHeight());
        } else {
            return super.getPreferredSize();
        }
    }
}
```

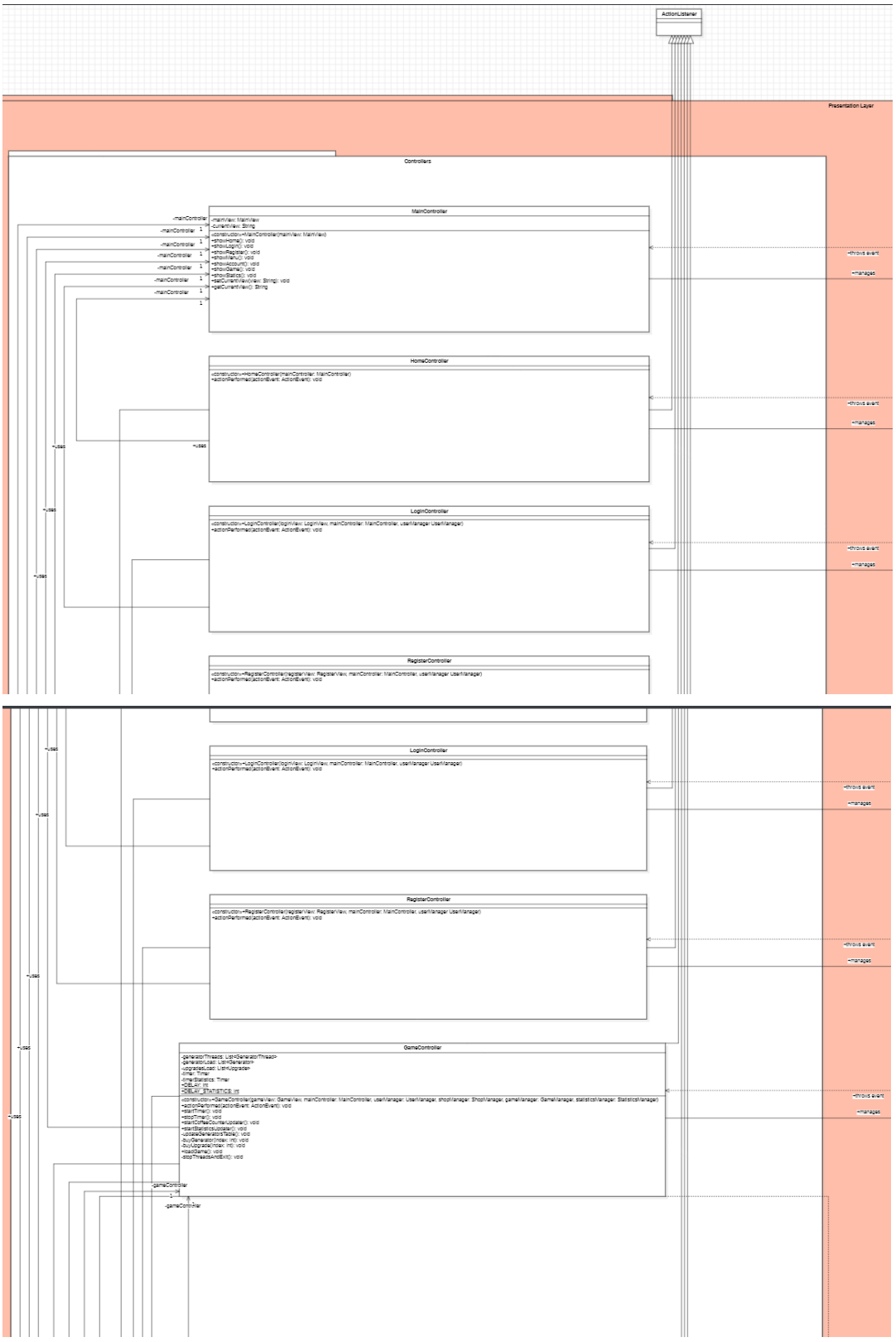
3. Diagrama de clases

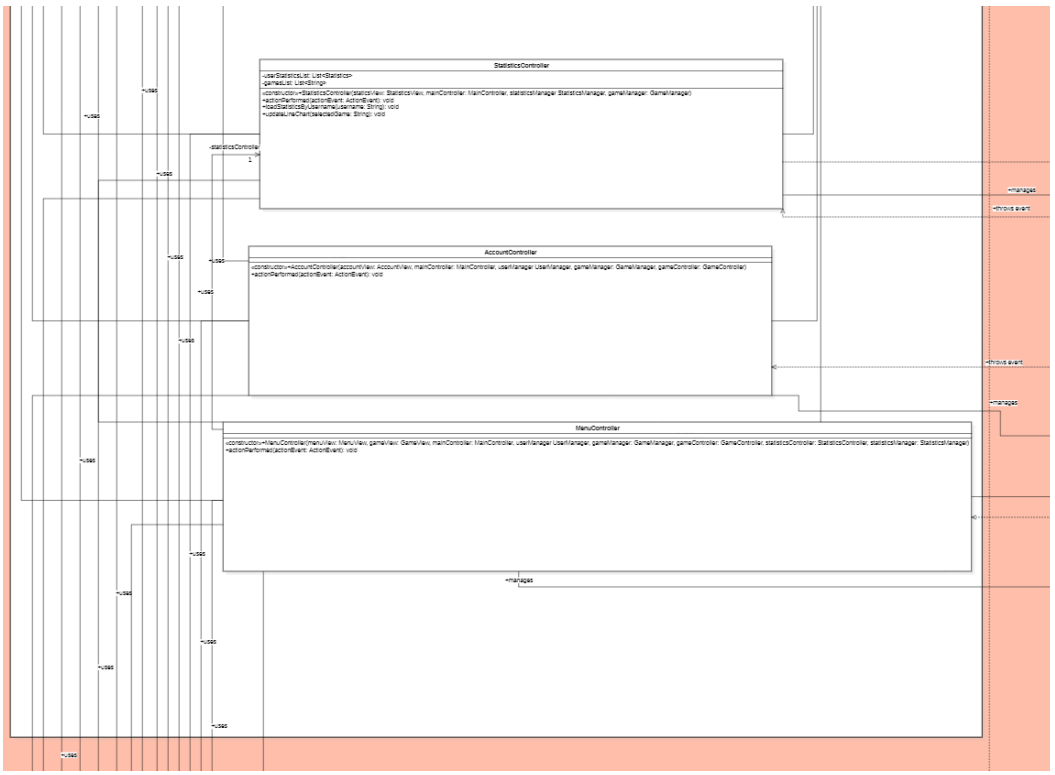
Link descarga diagrama: UML_Project_D_Coffee-Clicker_Final.mdj

Paquete general Presentation:

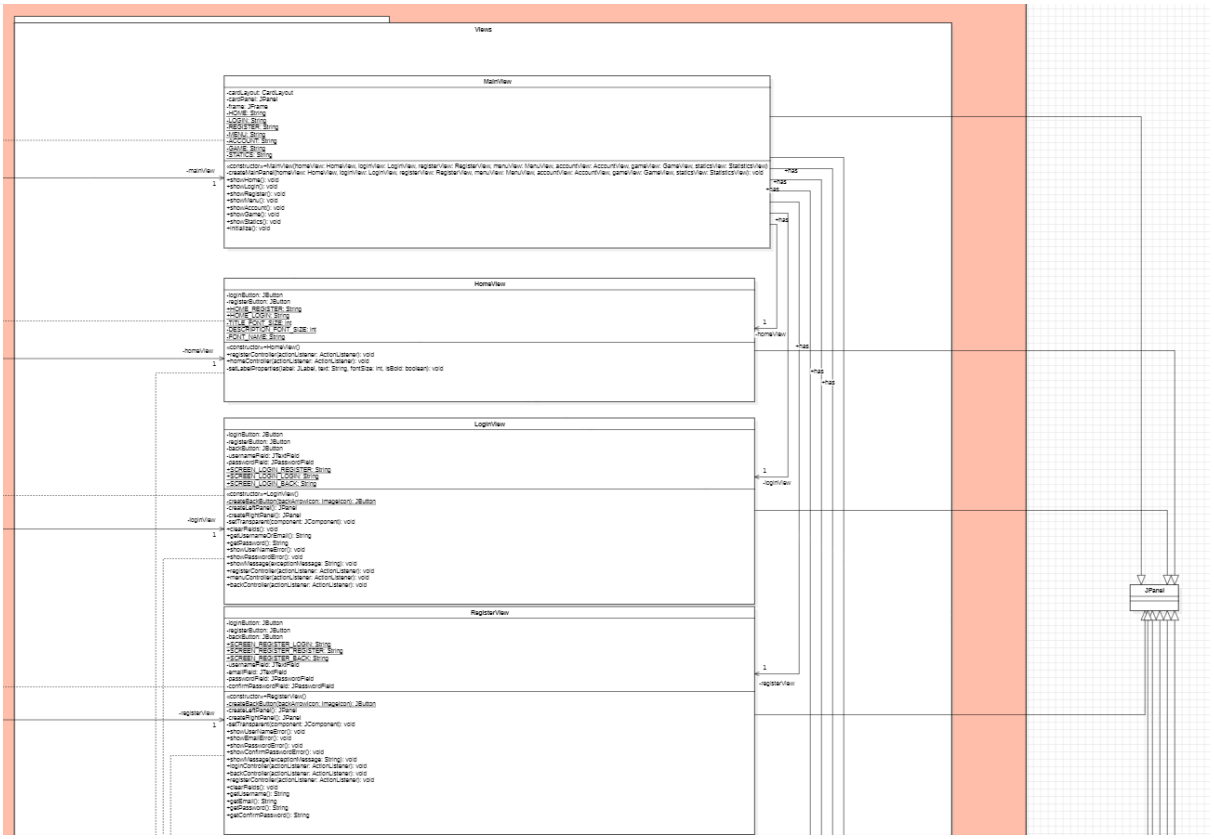


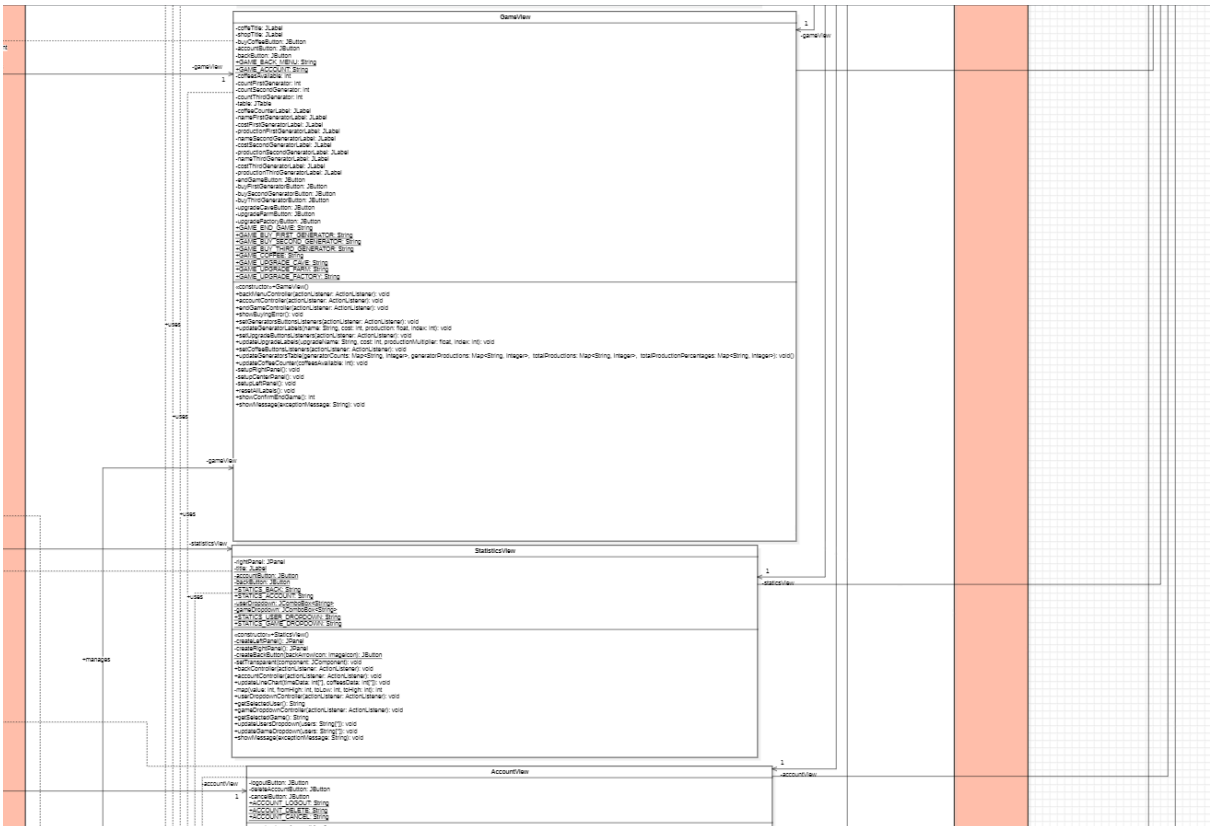
Presentation paquete Controllers:

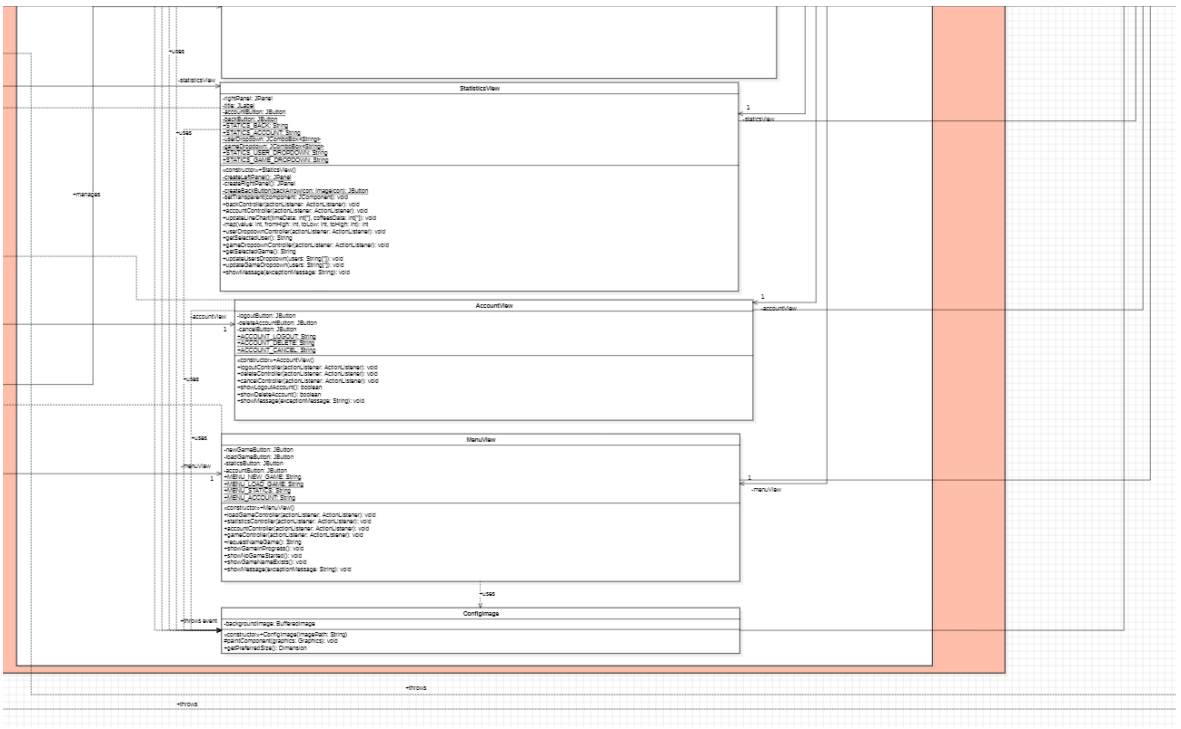



StatisticsController
-userStatisticsList: List<Statistics>
-gameList: List<String>
+constructor+StatisticsController(statsView: StatisticsView, mainController: MainController, statisticsManager: StatisticsManager, gameManager: GameManager)
+actionPerformed(actionEvent: ActionEvent): void
+loadStatistics(byUsername(username: String): void
+updateLineChart(selectedGame: String): void
-statisticsController
1
+uses
+manages
+throws event
AccountController
+constructor+AccountController(accountView: AccountView, mainController: MainController, userManager: UserManager, gameManager: GameManager, gameController: GameController)
+actionPerformed(actionEvent: ActionEvent): void
+uses
+throws event
+manages
MenuController
+constructor+MenuController(menuView: MenuView, gameView: GameView, mainController: MainController, userManager: UserManager, gameManager: GameManager, gameController: GameController, statisticsController: StatisticsController, statisticsManager: StatisticsManager)
+actionPerformed(actionEvent: ActionEvent): void
+uses
+manages

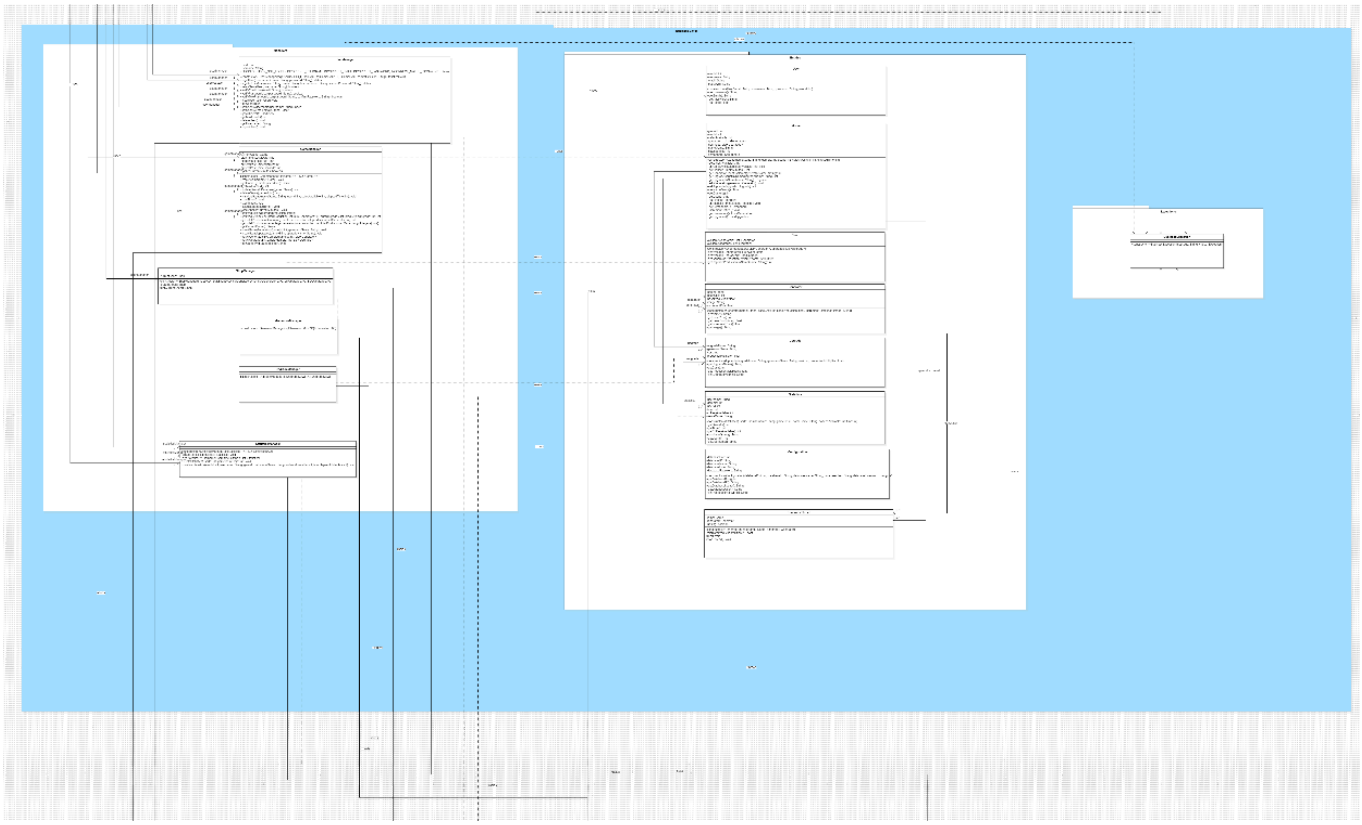
Presentation paquete views:



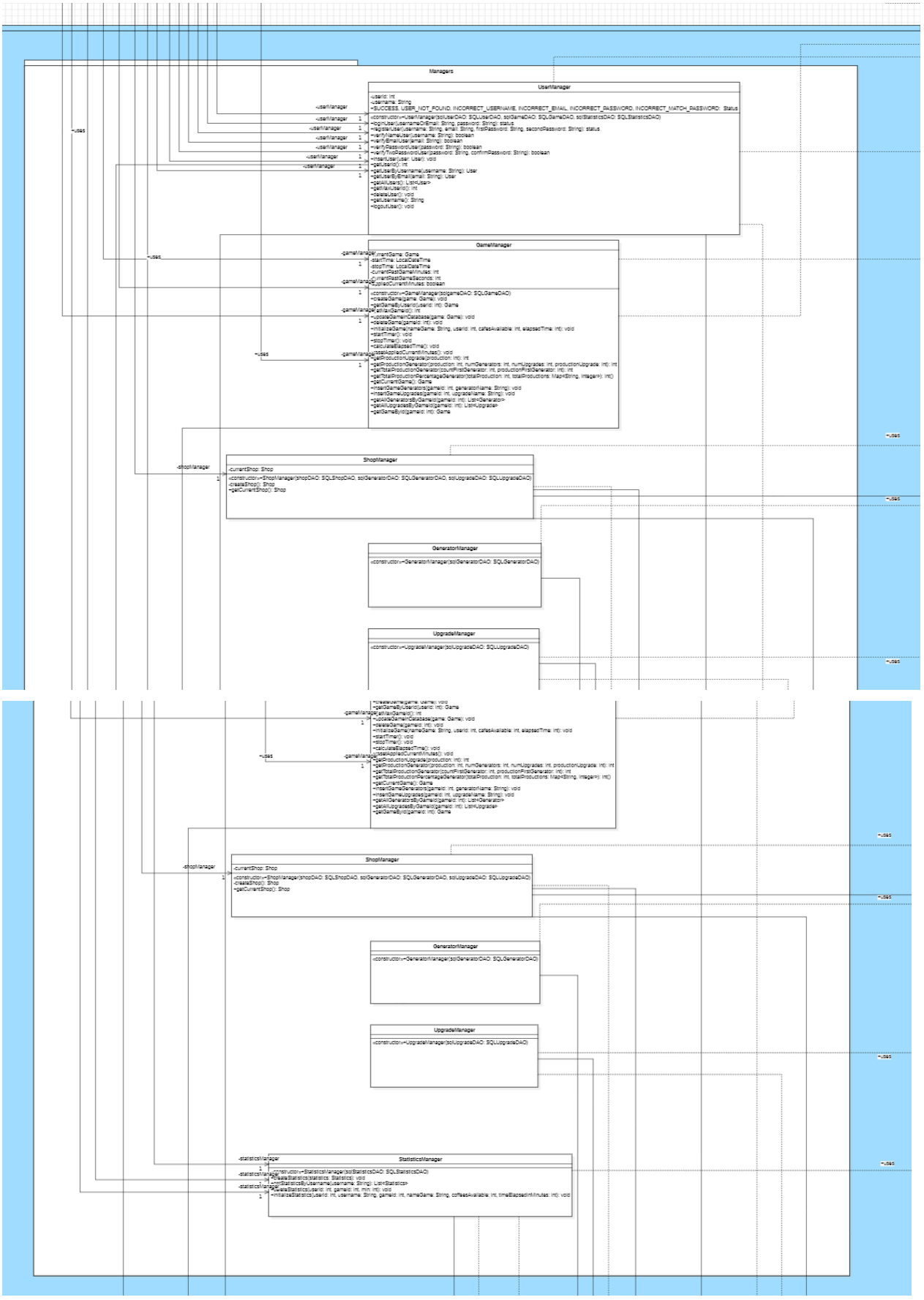



StatisticsView
AccountView
ManuView
ConfigImage
+uses
-statisticsView
-accountView
-menuView
+manages
+throws

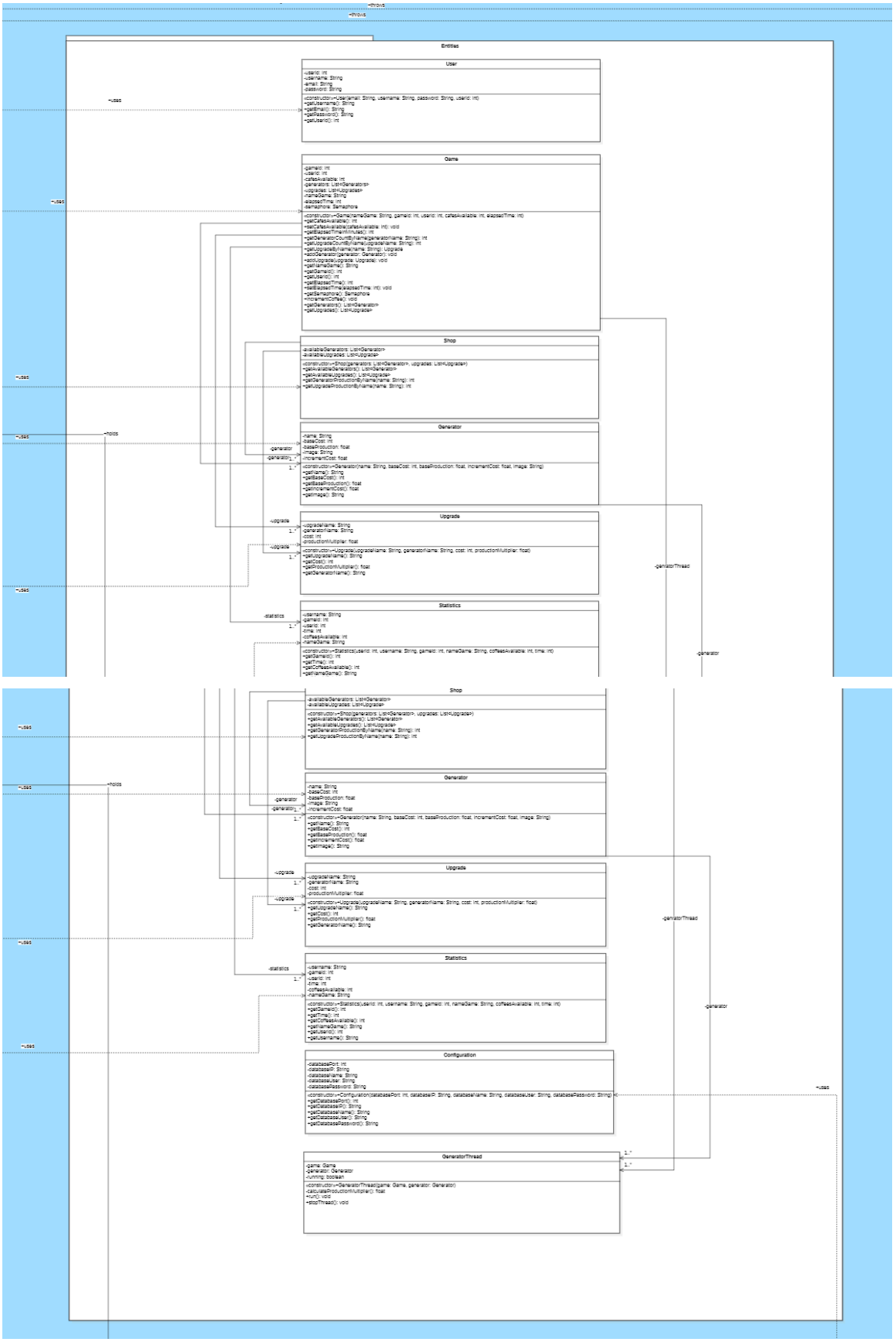
Paquete general business:



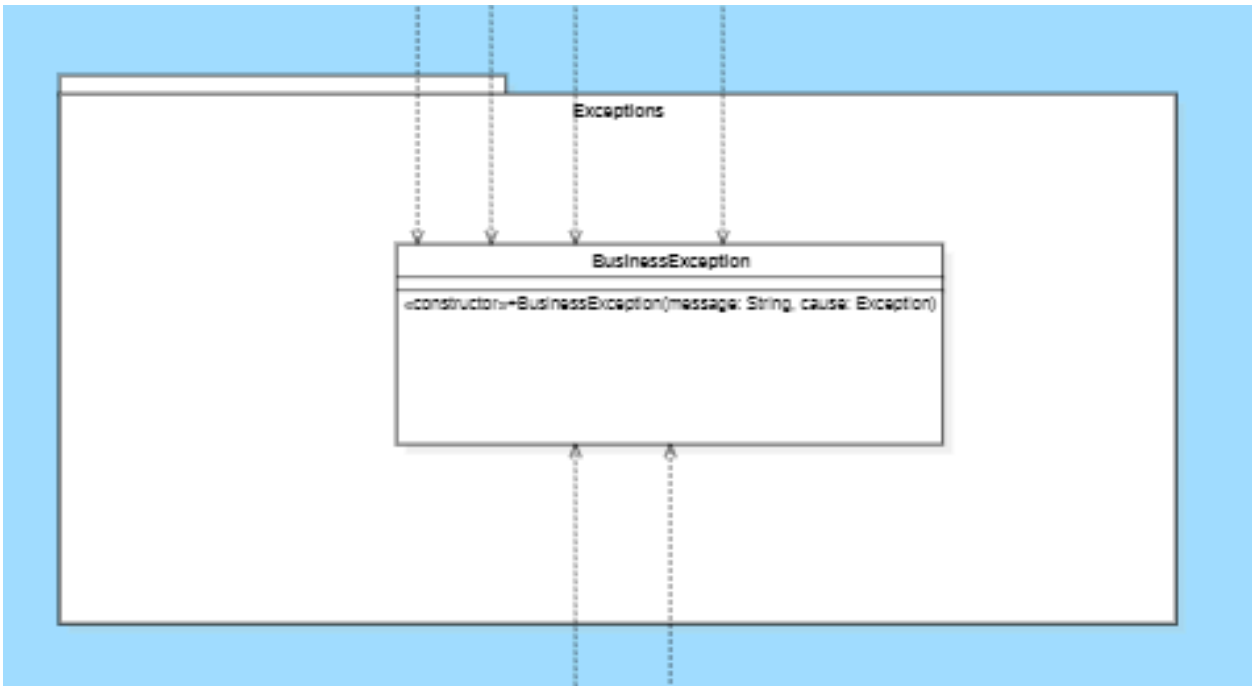
Business paquete Managers:



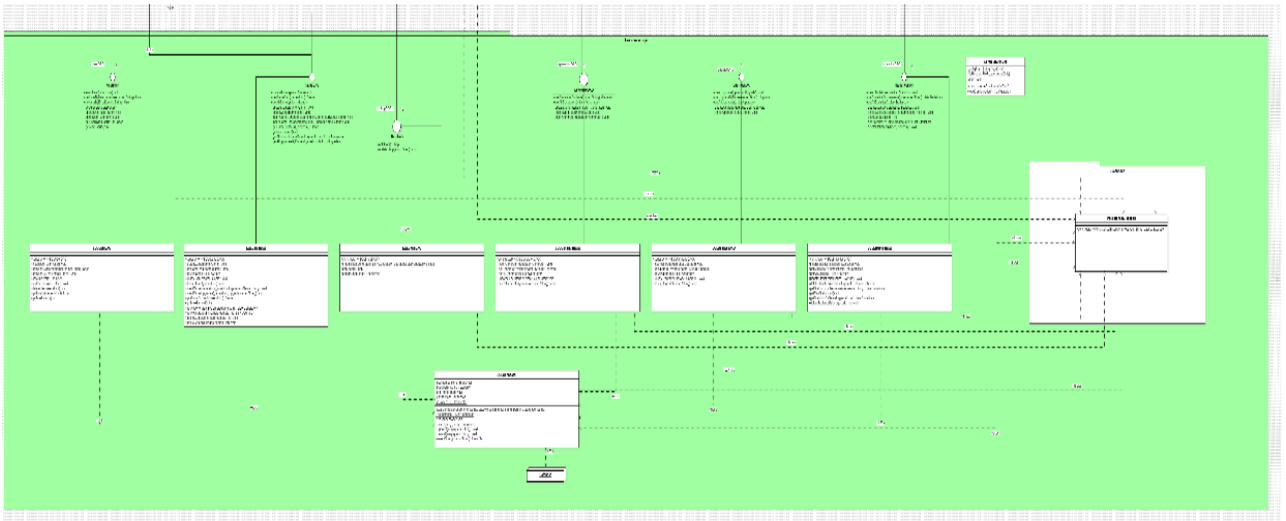
Business paquete Entities:



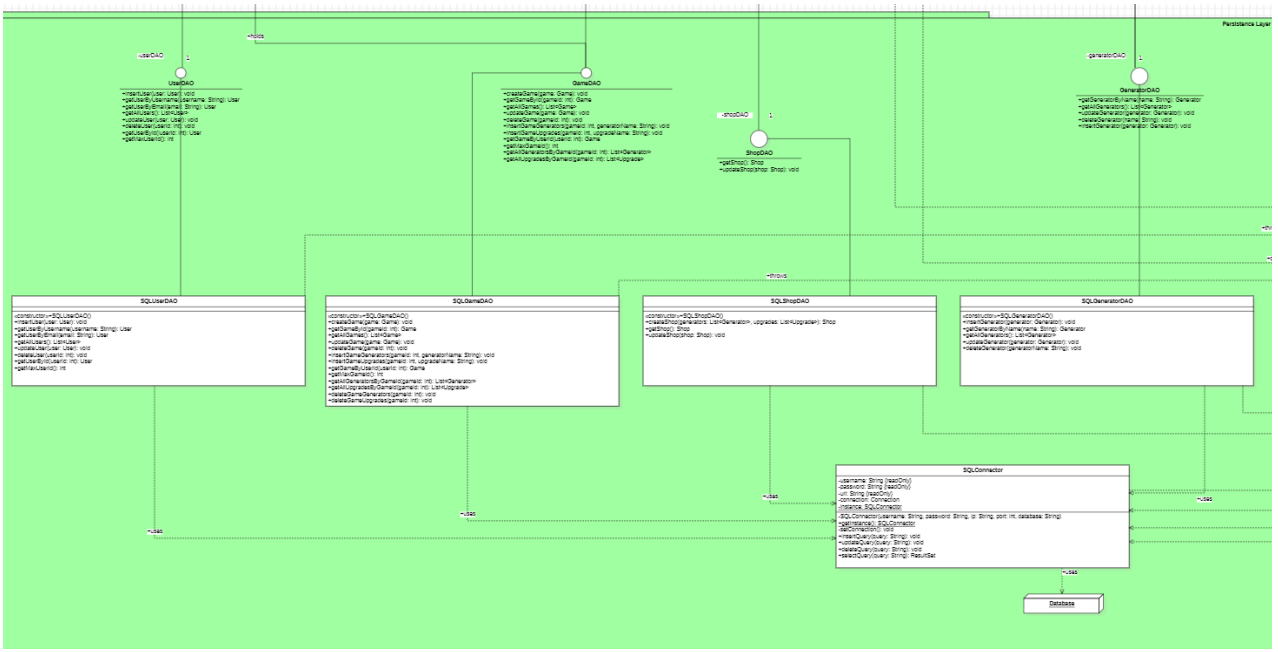
Business paquete Exceptions:

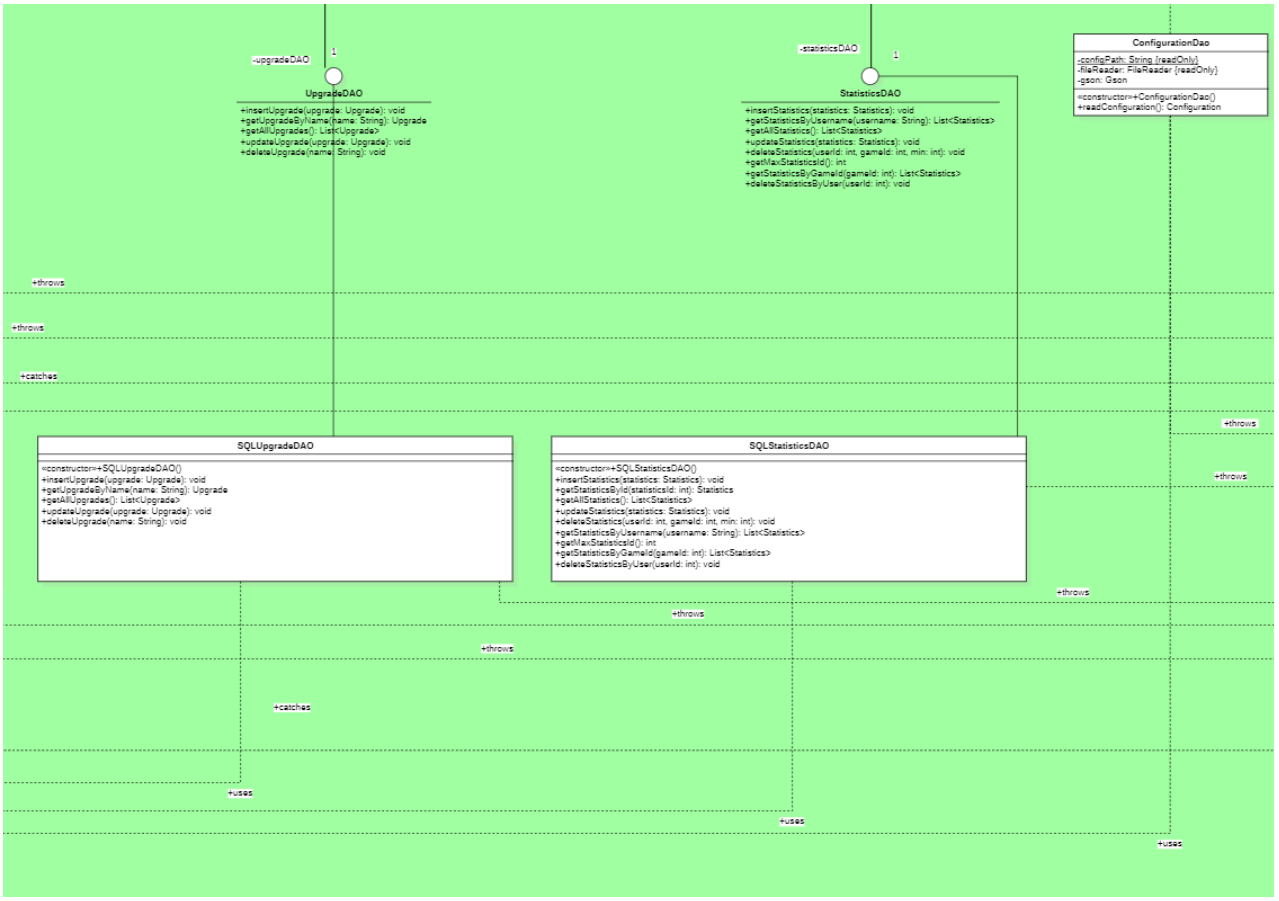


Paquete general persistance:

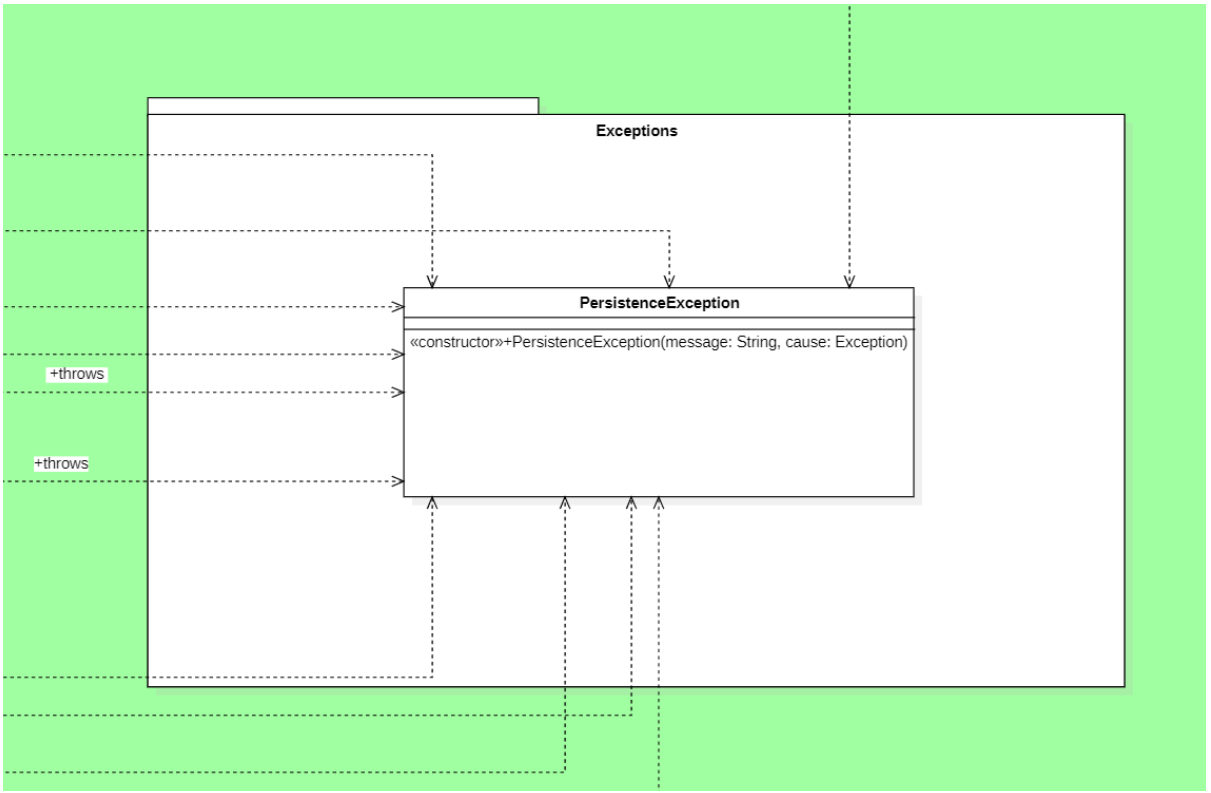


Paquete general ampliado:





Persistence paquete Exceptions:



A la hora de hacer el diagrama de clases, usamos el método de "Layered Architecture", conocido como arquitectura por capas o por niveles. Esto nos servirá para ir pudiendo agrupar las distintas funciones del programa, las cuales son paquetes que tienen su propio papel y funcionamiento en el código.

Estos paquetes son el de "presentation", el cual se encarga de tener todo lo que tiene que ver con la interfaz gráfica de la aplicación, es decir todas esas clases que tienen interacción con el usuario. El paquete de "presentation", ha sido dividido por dos sub paquetes, los cuales son el de "Controllers", que contienen todos los controladores del programa, y luego tenemos el sub paquete de "Views", el cual contiene todas las vistas del programa.

A continuación tenemos el paquete de "business", el cual contiene toda la lógica del programa, de crear, modificar, eliminar,etc, Es decir contiene todas las entidades que tienen la función de interactuar con los ficheros de persistencia. Este igual que el paquete de "presentation", ha sido dividido por dos sub paquetes, donde uno es el de "Entities", el cual contiene todas las entidades usadas en el programa, con sus atributos y métodos, y luego tenemos el segundo sub paquete que es el de "Managers", el cual contiene los managers de cada una de las entidades que tenemos.

Y por último tenemos el paquete de "persistance", el cual contiene los ficheros de persistencia. Cada uno de los ficheros de persistencia que hemos creado, han sido conectados a un "SQLConnector", el cual se encarga de hacer la conexión a la base de datos creada por nosotros. Esta base de datos que hemos creado se llama coffeeclicker_db.

Cabe resaltar que para el diagrama hemos hecho que las vistas y controladores de las mismas residirán en la capa de presentación, mientras que la capa de negocio hace de modelo. Con esto pretendemos mantener las vistas totalmente desacopladas del modelo, de forma que el modelo pueda trabajar con total independencia de la implementación utilizada para representar las vistas.

3.1. Paquete Presentation:

En este layer tenemos las views con sus respectivos controllers que gestionan las views. Cada controller tienen una asociación de generalización con el ActionListener, es decir, el controller está utilizando la herencia para usar el ActionListener por lo que hereda las capacidades de ActionListener, lo que le permite escuchar y responder a eventos de acción, como clics de botones o selecciones de menús, en la interfaz de usuario.

Hay un MainController y una MainView que controlan el resto de controllers y views respectivamente. Las views tienen una relación de dependencia con ConfigImage, las Views dependen de la clase ConfigImage para obtener información o funcionalidades relacionadas con la configuración de imágenes. Todos los controllers usan al UserManager para modificar las entidades. MainView tiene una relación de generalización con JFrame, esto significa que MainView es considerado un JFrame y permite que MainView tenga todas las características básicas de una ventana, como el diseño, la capacidad de contener otros componentes de la interfaz de usuario, la gestión de eventos y la capacidad de ser mostrada en pantalla. Para el resto de Views el caso es lo mismo pero con JPanel, las otras views son tratadas como un JPanel y por ende esto implica que MenuView puede contener otros componentes de interfaz de usuario, como botones y etiquetas o campos de texto.

3.2. Paquete Business:

Cada Manager está conectado a su respectivo DAO mediante una interfaz y luego ese DAO emplea la clase SQLConnector para modificar la Base de datos. Cada Manager modifica su entidad correspondiente como por ejemplo el UserManager modifica la entidad User, el GameManager modifica la entidad Game, etc. Game tiene una relación de asociación directa con Generator, Upgrade y Statistics lo cual significa que utiliza instancias de dichas clases y cada una es usada por el game para realizar acciones. Generator se ocupa de generar coffees en el Game, Upgrades se usa para multiplicar la generación de los coffees y statistics se ocupa de calcular los porcentajes para la tabla. La clase Shop tiene una relación de asociación directa con Generator y Upgrade, Shop utiliza estas clases para mostrar las que hay y buscar las que hay disponibles. Para gestionar las excepciones tenemos la clase businessExceptions, que se relaciona con userManager, gameManager y statisticsManager del paquete business y con gameController y statisticsController del paquete Presentation.

3.3. Paquete Persistence:

En este paquete se encuentran las interfaces que conectan los managers con los DAOs que estos tienen una relación de dependencia con la clase SQLConnector que establece conexión con la base de datos y realiza consultas de inserción, actualización, selección y eliminación de datos en la base de datos. Los DAOs proporcionan métodos para realizar operaciones como crear, updatear, eliminar e insertar que los managers usarán para actualizar las clases que administran. Las interfaces definen los métodos que los DAOs pueden implementar. Para gestionar las excepciones del paquete, hemos añadido la clase persistanceExceptions, que se relaciona con statisticsManager y userManager del paquete de Business y con configurationDAO y las clases SQL que se relacionan directamente con la base de datos.

4. Metodología de desarrollo

El proyecto consiste de un total de cuatro sprints, durante los sprints en clase el tutor monitorizará el trabajo realizado y ayudará a la completación del proyecto. Por lo general, durante los sprints todos hemos trabajado al mismo tiempo en la misma cosa pero más tarde sobre finales de primer sprint y durante el segundo sprint nos hemos dividido las tareas aunque nos hemos ayudado mutuamente. A continuación, se listará las tareas de cada miembro en cada sprint, cabe resaltar que se acabó antes de tiempo el proyecto por lo que el trabajo realizado en ciertos sprints es menor que en otros.

En el primer sprint, todos trabajamos de manera conjunta en el figma haciendo el prototipo de la UI y Gabriel junto con Marc empezaron el diagrama de clases. Una vez acabado el figma David se puso con el código y en este sprint realizó toda la parte de la GUI pero sin la lógica de funcionamiento del programa y Pau asistió en el código.

En el segundo sprint, Gabriel empezó la memoria e hizo la explicación general, la explicación del figma, Java y diagrama de clases. David acabó la implementación del Java con asistencia de Pau, Marc y Gabriel.

Las herramientas empleadas para la realización del proyecto son:

- Jira: Se ha utilizado esta herramienta para organizar el proyecto y ver las tareas en curso, facilitando la gestión y seguimiento del progreso del equipo.

- GitHub: Se ha usado esta herramienta para subir el proyecto y compartirlo con los miembros del equipo, permitiendo una colaboración eficiente y un control de versiones efectivo.

- IntelliJ: Esta herramienta ha sido empleada para desarrollar la parte del código y el proyecto, brindando un entorno integrado que optimiza el proceso de desarrollo y depuración.

- XAMPP: Se ha usado para conectarse a la base de datos, proporcionando un entorno de servidor local que simplifica el desarrollo y prueba de la aplicación.

- WORD: Se ha utilizado para realizar la memoria, ofreciendo un medio para documentar de manera estructurada y profesional los detalles del proyecto.

- Figma: Se ha usado esta herramienta para hacer un sketch de la GUI, facilitando el diseño y la visualización de la interfaz de usuario de manera colaborativa.

- PhpMyAdmin: Se ha empleado esta herramienta para crear la base de datos y hacer las consultas MySQL, permitiendo una gestión eficiente de la base de datos y la ejecución de consultas SQL de manera intuitiva.

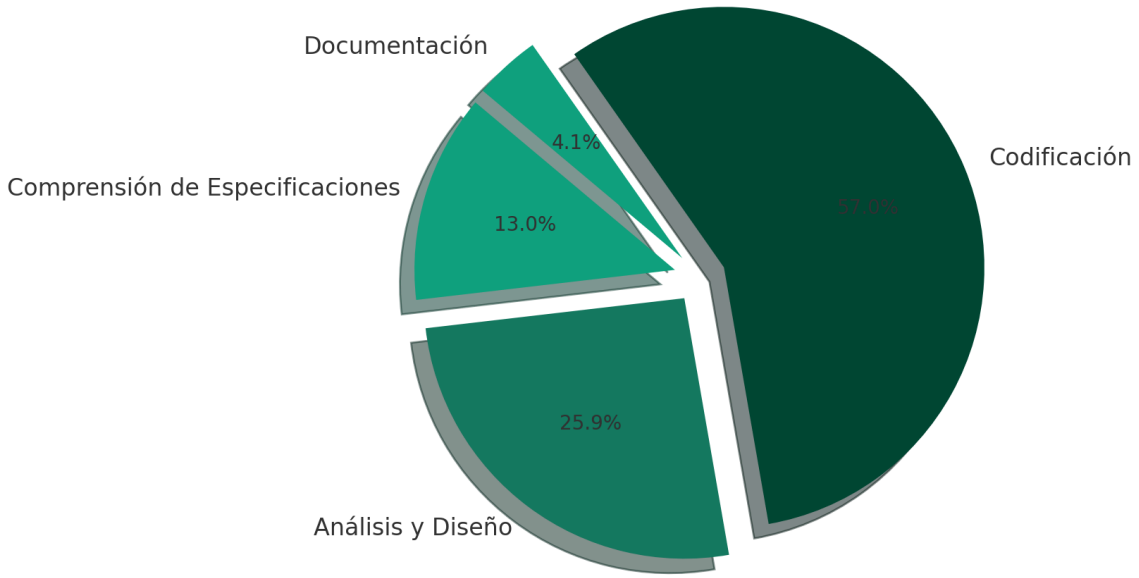
5. Costos de tiempo

La comprensión de las especificaciones fué relativamente deprisa y fluida. Debido a que todo está francamente bien explicado y ya tenemos cierta experiencia en resoluciones de prácticas, nos ha significado un tiempo estimado de 5 horas por integrante, es decir, 25 horas aproximadamente.

En cuanto al análisis y diseño, quizás nos ha tomado más tiempo de lo esperado ya que nos hemos encontrado con conceptos que no habíamos visto hasta ahora o que teníamos bastante gris, como ahora la implementación de la base de datos. Esto nos ha significado una breve prolongación del tiempo esperado a la dedicación al análisis del proyecto. Estimamos que el tiempo emprendido para obtener un análisis y diseño satisfactorio ronda en torno a las 50 horas.

La parte que nos ha requerido más tiempo, como era de esperar, ha sido la codificación. Especulamos un total de 100 a 120 horas entre todo el equipo.

Por último, el tiempo dedicado a la documentación rondaría las 8-10 horas aproximadamente.



Un total de: **193 horas**

6. Conclusiones

Este proyecto ha permitido al equipo adquirir y mejorar las habilidades de colaboración efectiva, haciendo un trabajo en equipo eficiente y efectivo. Todos los integrantes hemos contribuido activamente en las distintas fases del proyecto, lo que ha permitido una distribución equitativa del trabajo. Como equipo, compartimos una pasión por los videojuegos, y creemos que la elección de este proyecto ha sido la más idónea. Nos ha aportado una motivación y pasión extra que en cualquier otro proyecto, no hubiésemos tenido.

Nos hemos sentido cómodos con el método de enseñanza PBL (Project-Based Learning). Hemos sabido encarar y desenvolver todos los conceptos nuevos que nos ha ido proporcionando este proyecto, ya sea mediante herramienta externas o bien consultando al profesorado. Creemos que este método de aprendizaje nos ha hecho consolidar los conceptos de forma mucho más clara, a pesar de ser un poco más laborioso. Por ende, sentimos haber adquirido nuevas habilidades técnicas relacionadas con Java, el diseño de interfaces gráficas y gestión de base de datos. El uso de herramientas como intelliJ, XAMPP, Figma, Jira y GitHub ha sido crucial para el desarrollo y gestión del proyecto.

Evidentemente, cada concepto nuevo nos ha supuesto enfrentar un desafío nuevo, como los nombrados anteriormente, la implementación de la base de datos, la integración de la interfaz gráfica y la lógica del juego.

La planificación ha sido un poco más caótica, ya que teníamos una planificación proporcionada por la asignatura, con seguimiento incluido, pero la motivación y entusiasmo del equipo ha concluido en haber ido siempre un paso por delante de cara a los sprints y deadlines. Todos los integrantes del grupo hemos podido seguir el ritmo de forma cómoda, ha permitido finalizar el proyecto con tiempo y consolidar todos los conceptos eficientemente.
A lo largo del proyecto, el equipo ha aprendido a adaptarse a situaciones adversas y a ajustar las estrategias y métodos según fuera necesario.

Para concluir, resaltar y honorificar el esfuerzo y capacidad de trabajo en equipo de cada integrante del grupo que ha contribuido con la efectividad del proyecto concluyendo en un trabajo final muy satisfactorio. También agradecer al profesorado, incluyendo los becarios que han estado siempre dispuestos a solventar dudas.

7. Bibliografía

En esta sección se enumeran las páginas a las que hicimos referencia para mostrar dónde encontramos información que nos resultó útil durante el desarrollo del proyecto Coffee Clicker.

1. Estudy. Documentació sobre bases de dades. Basic database manual [en línea] [fecha de consulta: 17 marzo 2024]. Disponible en: https://estudy2324.salle.url.edu/pluginfile.php/26472/mod_folder/content/0/Basic%20database%20manual.pdf?forcedownload=1

2. Estudy. Documentació sobre bases de dades. Database manual for Java [en línea] [fecha de consulta: 17 marzo 2024]. Disponible en: https://estudy2324.salle.url.edu/pluginfile.php/26472/mod_folder/content/0/Database%20manual%20for%20Java.pdf?forcedownload=1

3. Estudy. Documentació sobre bases de dades. DAOs Example [en línea] [fecha de consulta: 17 marzo 2024]. Disponible en: https://estudy2324.salle.url.edu/pluginfile.php/26472/mod_folder/content/0/DAOs%20Example.zip?forcedownload=1

4. Estudy. Documentació sobre bases de dades. mysql-connector-java-8.0.23 [en línea] [fecha de consulta: 19 marzo 2024]. Disponible en: https://estudy2324.salle.url.edu/pluginfile.php/26472/mod_folder/content/0/mysql-connector-java-8.0.23.jar?forcedownload=1

5. Estudy. Integració de Git amb IntelliJ IDEA [en línea] [fecha de consulta: 5 abril 2024].Disponible en: https://estudy2324.salle.url.edu/mod/resource/view.php?id=17004

6. Oracle. Trail: Creating a GUI With Swing [en línea] [fecha de consulta: 5 abril 2024]. Disponible en: https://docs.oracle.com/javase/tutorial/uiswing/index.html

7. Bro Code. Java GUI: Full Course [en línea] [fecha de consulta: 8 abril 2024]. Disponible en: https://www.youtube.com/watch?v=Kmgo00avvEw